

CLOTHES SHOP

The database I created works as a complete structure for managing and monitoring several parts of a clothes business, such as client data, inventory, orders, and items inside those orders.

This database structure gives the clothes store information about sales, popular products, and client preferences while supporting fast inventory management, order processing, and customer relationship management.

1. CUSTOMERS Table: stores information about customers who interact with the clothing store.

Attributes:

- customer_id: Unique identifier for each customer;
- first_name: Customer's first name;
- last_name: Customer's last name;
- phone: Customer's contact phone number;
- email: Customer's email address;

2. CATEGORIES Table: represents the various categories of clothing available in the store.

Attributes:

- category_id: Unique identifier for each category.
- category_name: Name of the clothing category.

3. PRODUCTS Table: stores information about the products available in the store, including their prices and categories.

Attributes:

- product_id: Unique identifier for each product.
- product_name: Name of the product.
- price: Price of the product.
- category_id: Reference to the category the product belongs to through category_id.

4. ORDERS Table: tracks information about orders made by customers, including the date of purchase.

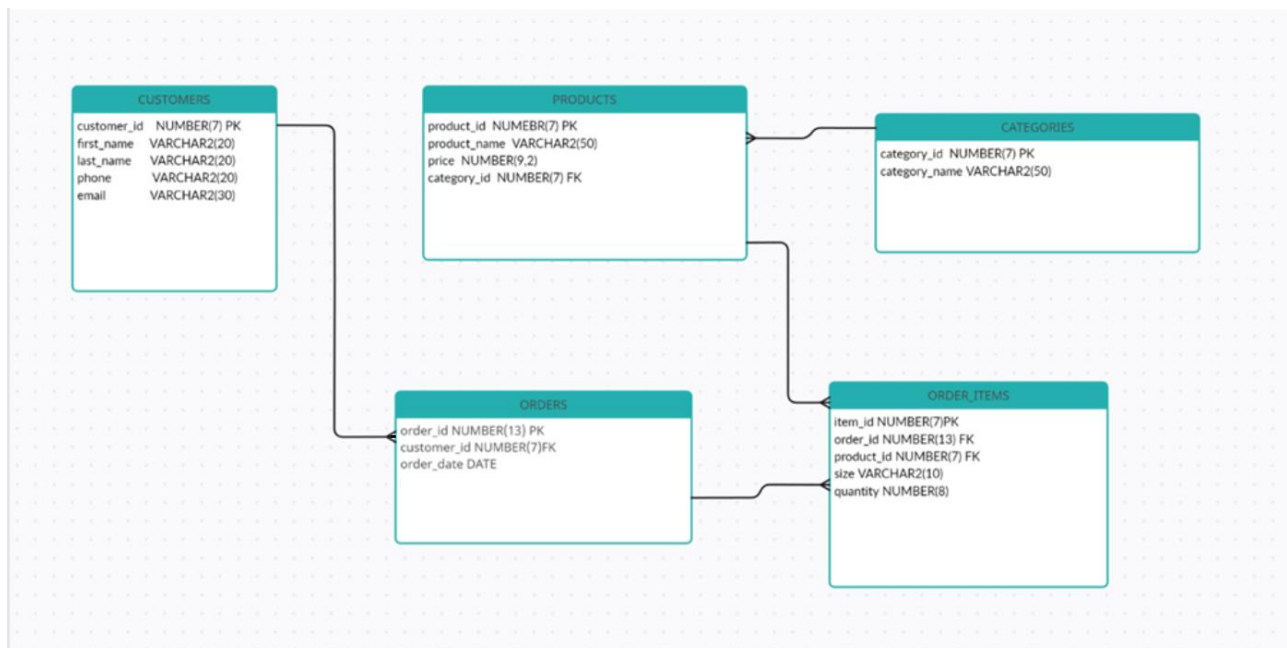
Attributes:

- order_id: Unique identifier for each order.
- customer_id: Reference to the customer who placed the order through customer_id.
- order_date: Date when the order was placed.

5. ORDER_ITEMS Table: stores details of items within orders, including the product, quantity, and size.

Attributes:

- item_id: Unique identifier for each item in an order.
- order_id: Reference to the order the item belongs to through order_id.
- product_id: Reference to the product being ordered through product_id.
- order_size: Size specification of the product in the order.
- quantity: Quantity of the product ordered.



1. Constructing the database: tables and constraints, exemplifying DDL statements on the tables: CREATE, ALTER, DROP (minimum 7 statements)

```
1  -- Create CUSTOMERS Table
2  CREATE TABLE CUSTOMERS (
3      customer_id NUMBER(7) CONSTRAINT pk_customer PRIMARY KEY,
4      first_name VARCHAR2(20),
5      last_name VARCHAR2(20),
6      phone VARCHAR2(20),
7      email VARCHAR2(30)
8  );
```

Results	Explain	Describe	Saved SQL	History
Table created.				
0.05 seconds				

```
-- Create CATEGORIES Table
CREATE TABLE CATEGORIES (
    category_id NUMBER(7) CONSTRAINT pk_category PRIMARY KEY,
    category_name VARCHAR2(50)
);
```

```
-- Create PRODUCTS Table
CREATE TABLE PRODUCTS (
    product_id NUMBER(7) CONSTRAINT pk_product PRIMARY KEY,
    product_name VARCHAR2(50),
    price NUMBER(9,2),
    category_id NUMBER(7),
    CONSTRAINT fk_category FOREIGN KEY (category_id) REFERENCES CATEGORIES(category_id)
);
```

```
-- Create ORDERS Table
CREATE TABLE ORDERS (
  order_id NUMBER(7) CONSTRAINT pk_order PRIMARY KEY,
  customer_id NUMBER(7),
  order_date DATE,
  CONSTRAINT fk_customer FOREIGN KEY (customer_id) REFERENCES CUSTOMERS(customer_id)
);
```

```
-- Create ORDER_ITEMS Table
CREATE TABLE ORDER_ITEMS (
  item_id NUMBER(10) CONSTRAINT pk_orderitems PRIMARY KEY,
  order_id NUMBER(7),
  product_id NUMBER(7),
  order_size VARCHAR2(10),
  quantity NUMBER(8),
  CONSTRAINT fk_order FOREIGN KEY (order_id) REFERENCES ORDERS(order_id),
  CONSTRAINT fk_product FOREIGN KEY (product_id) REFERENCES PRODUCTS(product_id)
);
```

```
-- Adding unique constraint for email in CUSTOMERS TABLE
ALTER TABLE CUSTOMERS
ADD CONSTRAINT unique_email UNIQUE (email);
```

Results	Explain	Describe	Saved SQL	History
---------	---------	----------	-----------	---------

Table altered.

```
-- Adding not null constraint for price in PRODUCTS TABLE
ALTER TABLE PRODUCTS
MODIFY price NUMBER(10, 2) NOT NULL;
```

Results	Explain	Describe	Saved SQL	History
---------	---------	----------	-----------	---------

Table altered.

```
-- Dropping tables
DROP TABLE ORDER_ITEMS;
DROP TABLE ORDERS;
DROP TABLE PRODUCTS;
DROP TABLE CATEGORIES;
DROP TABLE CUSTOMERS;
```

2. Using DML statements: INSERT, UPDATE, DELETE, and MERGE (minimum 10). Mandatory, a table must contain a row with the name of the student and a print screen of that row must be included in the project. In case the print screen is missing the grade will be 2.

```
--insert into the customers table
INSERT INTO CUSTOMERS (customer_id,first_name,last_name,phone,email)
VALUES (1,'Andreea','Ionescu','0722976429','andreea@gmail.com');

INSERT INTO CUSTOMERS (customer_id,first_name,last_name,phone,email)
VALUES(2,'Andrei','Petrescu','0724369744','andrei123@gmail.com');

INSERT INTO CUSTOMERS (customer_id,first_name,last_name,phone,email)
VALUES(3,'Ana-Maria','Popescu','0737827252','ana.maria24@gmail.com');

INSERT INTO CUSTOMERS (customer_id,first_name,last_name,phone,email)
VALUES(4,'Bianca','Petrescu','0727847233','biaa.petrescu@gmail.com');

INSERT INTO CUSTOMERS (customer_id,first_name,last_name,phone,email)
VALUES(5,'Emilia','Anghel','0733845533','emilia.anghel@gmail.com');
```

Results	Explain	Describe	Saved SQL	History
1 row(s) inserted.				

CUSTOMERS

Columns

Data

Indexes

Constraints

Grants

Statistics

Triggers

Dependencies

DDL

Sample Queries

+ Insert Row

Columns...

Filter...

Count Rows

Load Data

Download

Refresh

	CUSTOMER_ID	FIRST_NAME	LAST_NAME	PHONE	EMAIL
	3	Ana-Maria	Popescu	0737827252	ana.maria24@gmail.com
	4	Bianca	Petrescu	0727847233	biaa.petrescu@gmail.com
	5	Emilia	Anghel	0733845533	emilia.anghel@gmail.com
	2	Andrei	Petrescu	0724369744	andrei123@gmail.com
	1	Andreea	Ionescu	0722976429	andreea@gmail.com

A row with the name of the student !

	CUSTOMER_ID	FIRST_NAME	LAST_NAME	PHONE	EMAIL
	3	Ana-Maria	Popescu	0737827252	ana.maria24@gmail.com

```
--insert for categories table
INSERT INTO CATEGORIES(category_id,category_name)
VALUES(1,'T-shirts');

INSERT INTO CATEGORIES(category_id,category_name)
VALUES(2,'Dresses');

INSERT INTO CATEGORIES(category_id,category_name)
VALUES(3,'Jeans');

INSERT INTO CATEGORIES(category_id,category_name)
VALUES(4,'Tops');

INSERT INTO CATEGORIES(category_id,category_name)
VALUES(5,'Skirts');
```

CATEGORIES		
Columns	Data	Indexes Constraints Grants Statistics Triggers Dependencies DDL Sample Queries
+ Insert Row Columns... Filter... Count Rows Load Data Download Refresh		
	CATEGORY_ID	CATEGORY_NAME
	2	Dresses
	5	Skirts
	1	T-shirts
	3	Jeans
	4	Tops

```
--insert for products table
INSERT INTO PRODUCTS(product_id,product_name,price,category_id)
VALUES(220,'Oversize T-shirt',89.99,1);

INSERT INTO PRODUCTS(product_id,product_name,price,category_id)
VALUES(230,'Long dress',159.99,2);

INSERT INTO PRODUCTS(product_id,product_name,price,category_id)
VALUES(240,'Denim jeans',129.99,3);

INSERT INTO PRODUCTS(product_id,product_name,price,category_id)
VALUES(250,'Crop top',99.99,4);

INSERT INTO PRODUCTS(product_id,product_name,price,category_id)
VALUES(260,'Black dress',209.99,2);

INSERT INTO PRODUCTS(product_id,product_name,price,category_id)
VALUES(270,'Black T-shirt',60.99,1);

INSERT INTO PRODUCTS(product_id,product_name,price,category_id)
VALUES(280,'Pink skirt',139.99,5);

INSERT INTO PRODUCTS(product_id,product_name,price,category_id)
VALUES(290,'Silver top',259.99,4);
```

PRODUCTS				
Columns	Data	Indexes	Constraints	Grants Statistics Triggers Dependencies DDL Sample Queries
+ Insert Row Columns... Filter... Count Rows Load Data Download Refresh				
	PRODUCT_ID	PRODUCT_NAME	PRICE	CATEGORY_ID
	240	Denim jeans	129.99	3
	220	Oversize T-shirt	89.99	1
	280	Pink skirt	139.99	5
	290	Silver top	259.99	4
	230	Long dress	159.99	2
	260	Black dress	209.99	2
	250	Crop top	99.99	4
	270	Black T-shirt	60.99	1

```
--insert for orders table
INSERT INTO ORDERS(order_id,customer_id,order_date)
VALUES(223,1,TO_DATE('2023-02-20', 'YYYY-MM-DD'));

INSERT INTO ORDERS(order_id,customer_id,order_date)
VALUES(234,2,TO_DATE('2023-03-12','YYYY-MM-DD'));

INSERT INTO ORDERS(order_id,customer_id,order_date)
VALUES(245,3,TO_DATE('2023-04-10','YYYY-MM-DD'));

INSERT INTO ORDERS(order_id,customer_id,order_date)
VALUES(255,4,TO_DATE('2023-03-20', 'YYYY-MM-DD'));

INSERT INTO ORDERS(order_id,customer_id,order_date)
VALUES(265,5,TO_DATE('2023-01-23', 'YYYY-MM-DD'));
```

ORDERS

Columns

Data

Indexes

Constraints

Grants

Statistics

Triggers

Dependencies

DDL

Sample Queries

+ Insert Row

Columns...

Filter...

Count Rows

Load Data

Download

Refresh

	ORDER_ID	CUSTOMER_ID	ORDER_DATE
	245	3	04/10/2023
	255	4	03/20/2023
	223	1	02/20/2023
	234	2	03/12/2023
	265	5	01/23/2023

```
--insert for order_items table
INSERT INTO ORDER_ITEMS(item_id,order_id,product_id,order_size,quantity)
VALUES(301,223,220,'S',1);

INSERT INTO ORDER_ITEMS(item_id,order_id,product_id,order_size,quantity)
VALUES(302,234,230,'L',2);

INSERT INTO ORDER_ITEMS(item_id,order_id,product_id,order_size,quantity)
VALUES(303,245,230,'M',1);

INSERT INTO ORDER_ITEMS(item_id,order_id,product_id,order_size,quantity)
VALUES(304,255,250,'L',2);

INSERT INTO ORDER_ITEMS(item_id,order_id,product_id,order_size,quantity)
VALUES(305,265,260,'S',1);
```


ORDER_ITEMS						
Columns Data Indexes Constraints Grants Statistics Triggers Dependencies DDL Sample Queries						
+ Insert Row Columns... Filter... Count Rows Load Data Download Refresh						
	ITEM_ID	ORDER_ID	PRODUCT_ID	ORDER_SIZE	QUANTITY	
	302	234	230	L	2	
	304	255	250	L	2	
	301	223	220	S	1	
	303	245	230	M	1	
	305	265	260	S	1	

```
--UPDATE
--update the costumer email
UPDATE CUSTOMERS
SET email='andreea222@gmail.com'
WHERE customer_id=1;
```

1	Andreea	Ionescu	0722976429	andreea222@gmail.com
---	---------	---------	------------	----------------------

```
--update the product price
UPDATE PRODUCTS
SET price='109.99'
WHERE product_id=220;
```

220	Overize T-shirt	109.99	1
-----	-----------------	--------	---

```
DELETE FROM PRODUCTS
WHERE product_id=240;
```

1 row(s) deleted.

0.05 seconds

```
DELETE FROM ORDERS
WHERE order_id=245;
```

1 row(s) deleted.

- I created a new table where there are update data from CUSTOMERS to do MERGE

```
--I created table "CUSTOMERS_UPDATE_INFO"  
CREATE TABLE CUSTOMERS_UPDATE_INFO (  
    customer_id NUMBER(7) CONSTRAINT pk_customer_backup PRIMARY KEY,  
    first_name VARCHAR2(20),  
    last_name VARCHAR2(20),  
    phone VARCHAR2(20),  
    email VARCHAR2(30)  
);
```

The initial table:

Results Explain Describe Saved SQL History				
CUSTOMER_ID	FIRST_NAME	LAST_NAME	PHONE	EMAIL
1	Andreea	Ionescu	0722976429	andreea@gmail.com
2	Andrei	Petrescu	0724369744	andrei.petrescu44@gmail.com
4	Bianca	Petrescu	0727847233	biancaa.petrescu34@gmail.com
3	Ana-Maria	Popescu	07224369744	ana.maria24@gmail.com

After MERGE:

Results Explain Describe Saved SQL History

5 row(s) updated.

0.04 seconds

Results Explain Describe Saved SQL History				
CUSTOMER_ID	FIRST_NAME	LAST_NAME	PHONE	EMAIL
1	Andreea	Ionescu	0722976429	andreea222@gmail.com
2	Andrei	Petrescu	0724369744	andrei123@gmail.com
4	Bianca	Petrescu	0727847233	biaa.petrescu@gmail.com
5	Emilia	Anghel	0733845533	emilia.anghel@gmail.com
3	Ana-Maria	Popescu	0737827252	ana.maria24@gmail.com

5 rows returned in 0.01 seconds Download

3. Diverse and relevant SELECT statements for the project theme (minimum 15) that will contain (combined) all of the following elements:

- >, =, <=, !=, IS NULL, LIKE, IN, BETWEEN, ANY, ALL;
- Joins (inner, outer);
- Using aggregate functions, GROUP BY, HAVING;
- Using single row functions (mandatory: TO_CHAR, TO_DATE, EXTRACT, SUBSTR, SYSDATE, DECODE, CASE, NVL);
- Using the CASE statement;
- Using UNION, MINUS, INTERSECT;
- Simple and correlated subqueries;
- CREATE/INSERT/UPDATE/DELETE + SELECT;
- Constructing and manipulating other objects of the database: views, indexes, synonyms and sequences;
- Hierarchical queries (CONNECT BY, PRIOR, LEVEL, SYS_CONNECT_BY_PATH);

```
--1.display all the products with the price between 100 and 200 and have the product_id 220 and 230 (using BETWEEN and IN)
SELECT * FROM PRODUCTS
WHERE price BETWEEN 100 AND 200 AND product_id IN(220,230);
```

Results Explain Describe Saved SQL History				
PRODUCT_ID	PRODUCT_NAME	PRICE	CATEGORY_ID	
220	Oversize T-shirt	109.99	1	
230	Long dress	159.99	2	

```
--2.display only the customers whose first_name end in 'a' and have the customer_id bigger than 1 (using > and LIKE)
SELECT * FROM CUSTOMERS
WHERE first_name LIKE '%a'
AND customer_id > 1;
```

CUSTOMER_ID	FIRST_NAME	LAST_NAME	PHONE	EMAIL
3	Ana-Maria	Popescu	0737827252	ana.maria24@gmail.com
4	Bianca	Petrescu	0727847233	biaa.petrescu@gmail.com
5	Emilia	Anghel	0733845533	emilia.anghel@gmail.com

```
--3.displays all order items whose quantity is 1 and whose product_id is different from 230 (using = and !=)
SELECT * FROM ORDER_ITEMS
WHERE quantity=1
AND product_id!= 230;
```

ITEM_ID	ORDER_ID	PRODUCT_ID	ORDER_SIZE	QUANTITY
301	223	220	S	1
305	265	260	S	1

2 rows returned in 0.01 seconds [Download](#)

```
--4.display product_id and price for PRODUCTS where the price is less than or equal to all prices in the PRODUCTS table(using <= and ALL)
SELECT product_id, price
FROM PRODUCTS
WHERE price <= ALL (SELECT price FROM PRODUCTS);
```

PRODUCT_ID	PRICE
270	60.99

```
--5.display all customers where the customer_id matches any customer_id present in the ORDERS table(using ANY)
SELECT * FROM CUSTOMERS WHERE customer_id = ANY (SELECT customer_id FROM ORDERS);
```

CUSTOMER_ID	FIRST_NAME	LAST_NAME	PHONE	EMAIL
3	Ana-Maria	Popescu	0737827252	ana.maria24@gmail.com
4	Bianca	Petrescu	0727847233	biaa.petrescu@gmail.com
1	Andreea	Ionescu	0722976429	andreea222@gmail.com
2	Andrei	Petrescu	0724369744	andrei123@gmail.com
5	Emilia	Anghel	0733845533	emilia.anghel@gmail.com

```
--6.display all customers whose email IS NULL (using IS NULL)
SELECT *
FROM CUSTOMERS
WHERE email IS NULL;
```

no data found

```
--7.display the products name and count the quantity of items
-- for each product from the PRODUCTS and ORDER_ITEMS tables,
--with the condition to be less than 2
--(using inner join, GROUP BY,HAVING)
```

```
SELECT p.product_name, COUNT(oi.quantity) AS quantity_of_products
FROM PRODUCTS p,ORDER_ITEMS oi
WHERE p.product_id = oi.product_id
GROUP BY p.product_name
HAVING COUNT(oi.quantity) < 2;
```

Results

Explain

Describe

Saved SQL

History

PRODUCT_NAME	QUANTITY_OF_PRODUCTS
Black dress	1
Oversize T-shirt	1
Crop top	1

3 rows returned in 0.02 seconds

Download

```
--8.displays information about products even
--those that do not have order_items
--(using left join)
```

```
SELECT p.product_id, p.product_name, oi.order_id, oi.quantity
FROM PRODUCTS p
LEFT JOIN ORDER_ITEMS oi ON p.product_id = oi.product_id;
```

PRODUCT_ID	PRODUCT_NAME	ORDER_ID	QUANTITY
230	Long dress	234	2
250	Crop top	255	2
220	Oversize T-shirt	223	1
230	Long dress	245	1
260	Black dress	265	1
290	Silver top	-	-
270	Black T-shirt	-	-
240	Denim jeans	-	-
280	Pink skirt	-	-
9 rows returned in 0.01 seconds Download			

```
--9.display the orders and their placing date in initial format and in 'Mon-DD-YYYY' format,
--and also display the orders placed in April AND March (using TO_CHAR and EXTRACT)
```

```
SELECT
  order_id,
  customer_id,
  order_date,
  TO_CHAR(order_date, 'Mon-DD-YYYY') AS new_format_order_date_2,
  EXTRACT(MONTH FROM order_date) AS extract_month
FROM
  ORDERS
WHERE
  EXTRACT(MONTH FROM order_date) IN (3, 4);
```

ORDER_ID	CUSTOMER_ID	ORDER_DATE	NEW_FORMAT_ORDER_DATE_2	EXTRACT_MONTH
245	3	04/10/2023	Apr-10-2023	4
255	4	03/20/2023	Mar-20-2023	3
234	2	03/12/2023	Mar-12-2023	3

```
--10.display the orders and their placing date on day 20 (using TO_DATE and EXTRACT)
SELECT
    order_id,
    customer_id,
    TO_DATE(order_date, 'MM/DD/YYYY') AS new_format_order_date
FROM
    ORDERS
WHERE
    EXTRACT(DAY FROM order_date) IN(20);
```

ORDER_ID	CUSTOMER_ID	NEW_FORMAT_ORDER_DATE
255	4	03/20/2023
223	1	02/20/2023

```
--11.Display the current date (using sysdate)
SELECT SYSDATE AS current_date FROM dual;
```

CURRENT_DATE
12/30/2023

```
--12.display the first two characters from the first_name column in the CUSTOMERS(using SUBSTR)
SELECT
    SUBSTR(first_name, 1, 2) AS substr_name
FROM CUSTOMERS;
```

SUBSTR_NAME
An
Bi
Em
An
An

```
--13. display the names of customers whose first_names begin with the letter A or letter B(using DECODE and SUBSTR)
SELECT
    first_name,
    DECODE(SUBSTR(first_name, 1, 1), 'A', 'Name with A', 'B', 'Name with B', 'Other') AS name_first_letter
FROM
    CUSTOMERS;
```

FIRST_NAME	NAME_FIRST_LETTER
Ana-Maria	Name with A
Bianca	Name with B
Emilia	Other
Andrei	Name with A
Andreea	Name with A

5 rows returned in 0.01 seconds [Download](#)

```
--14.display the price_status according to the price they have(using CASE)
SELECT
    product_id,
    CASE
        WHEN price < 100 THEN 'Affordable'
        ELSE 'Expensive'
    END AS price_status
FROM PRODUCTS;
```

PRODUCT_ID	PRICE_STATUS
240	Expensive
220	Expensive
280	Expensive
290	Expensive
230	Expensive
260	Expensive
250	Affordable
270	Affordable

```
--15. display 'Not Available' whenever the phone number of customers is NULL (using NVL)
SELECT
    customer_id,
    first_name,
    last_name,
    NVL(phone, 'Not Available') AS phone_status
FROM CUSTOMERS;
```

CUSTOMER_ID	FIRST_NAME	LAST_NAME	PHONE_STATUS
3	Ana-Maria	Popescu	0737827252
4	Bianca	Petrescu	0727847233
5	Emilia	Anghel	0733845533
2	Andrei	Petrescu	0724369744
1	Andreea	Ionescu	0722976429

```
-- 16.Display the products that have the price between 100 and 200, but don't have the category_id 2 (using MINUS)
SELECT category_id, product_name, price
FROM PRODUCTS
WHERE price BETWEEN 100 AND 200
MINUS
SELECT category_id, product_name, price
FROM PRODUCTS
WHERE category_id = 2;
```

CATEGORY_ID	PRODUCT_NAME	PRICE
1	Oversize T-shirt	109.99
3	Denim jeans	129.99
5	Pink skirt	139.99

```
--17. Calculate the Total Customers and Total Orders , (using UNION operator)
SELECT 'Total Customers' AS information , COUNT(*) AS total_count
FROM CUSTOMERS
UNION
SELECT 'Total Orders' AS information , COUNT(*) AS total_count
FROM ORDERS;
```

INFORMATION	TOTAL_COUNT
Total Customers	5
Total Orders	5

```
--18. Display the customers who have placed orders
--in the current year and they have at least have an order(using INTERSECT)
SELECT customer_id
FROM ORDERS
WHERE EXTRACT(YEAR FROM order_date) = EXTRACT(YEAR FROM SYSDATE)
INTERSECT
SELECT customer_id
FROM ORDERS
GROUP BY customer_id
HAVING COUNT(order_id) > 0;
```

CUSTOMER_ID
1
2
3
4
5


```
--19.display the products that haven't been ordered(using simple subquery)
SELECT product_id, product_name
FROM PRODUCTS
WHERE product_id NOT IN (SELECT DISTINCT product_id FROM ORDER_ITEMS);
```

PRODUCT_ID	PRODUCT_NAME
290	Silver top
270	Black T-shirt
240	Denim jeans
280	Pink skirt

```
--21..Update the price of the product with product_id=260 and
-- then display the updated details(using SELECT+UPDATE)
UPDATE PRODUCTS
SET price = 309.99
WHERE product_id = 260;
SELECT *
FROM PRODUCTS
WHERE product_id = 260;
```

PRODUCT_ID	PRODUCT_NAME	PRICE	CATEGORY_ID
260	Black dress	309.99	2

```
--22.Create a view to display customer information along with order details(using view)
CREATE VIEW CustomerOrderDetails AS
SELECT c.customer_id, c.first_name, c.last_name, o.order_id, o.order_date
FROM CUSTOMERS c
LEFT JOIN ORDERS o ON c.customer_id = o.customer_id;

SELECT * FROM CustomerOrderDetails;
```

CUSTOMERORDERDETAILS

Columns

Code

Data

Errors

Grants

Dependencies

DDL

Columns...

Filter...

Count Rows

Download

Refresh

CUSTOMER_ID	FIRST_NAME	LAST_NAME	ORDER_ID	ORDER_DATE
3	Ana-Maria	Popescu	245	04/10/2023
4	Bianca	Petrescu	255	03/20/2023
1	Andreea	Ionescu	223	02/20/2023
2	Andrei	Petrescu	234	03/12/2023
5	Emilia	Anghel	265	01/23/2023

```
--23.Create an index on product_name column in PRODUCTS table(using index)
CREATE INDEX prod_name_idx ON PRODUCTS(product_name);
SELECT * FROM USER_INDEXES;
```

INDEX_NAME	INDEX_TYPE	TABLE_OWNER	TABLE_NAME	TABLE_TYPE	UNIQUENESS	COMPRESSION	PREFIX_LENGTH	TABLESPACE_NAME	INL_TRANS
PK_CATEGORY	NORMAL	WKSP_SALUTT	CATEGORIES	TABLE	UNIQUE	DISABLED	-	APEX_BIGFILE_INSTANCE_TBSS	2
PK_CUSTOMER	NORMAL	WKSP_SALUTT	CUSTOMERS	TABLE	UNIQUE	DISABLED	-	APEX_BIGFILE_INSTANCE_TBSS	2
PK_ORDER	NORMAL	WKSP_SALUTT	ORDERS	TABLE	UNIQUE	DISABLED	-	APEX_BIGFILE_INSTANCE_TBSS	2
PK_ORDERITEMS	NORMAL	WKSP_SALUTT	ORDER_ITEMS	TABLE	UNIQUE	DISABLED	-	APEX_BIGFILE_INSTANCE_TBSS	2
PK_PRODUCT	NORMAL	WKSP_SALUTT	PRODUCTS	TABLE	UNIQUE	DISABLED	-	APEX_BIGFILE_INSTANCE_TBSS	2
PROD_NAME_IDX	NORMAL	WKSP_SALUTT	PRODUCTS	TABLE	NONUNIQUE	DISABLED	-	APEX_BIGFILE_INSTANCE_TBSS	2
UNIQUE_EMAIL	NORMAL	WKSP_SALUTT	CUSTOMERS	TABLE	UNIQUE	DISABLED	-	APEX_BIGFILE_INSTANCE_TBSS	2

7 rows returned in 0.20 seconds [Download](#)

```
--24.Create a synonym for the ORDER_ITEMS table (using synonym)
CREATE SYNONYM ITEMS FOR ORDER_ITEMS;
SELECT* FROM ITEMS;
```

ITEM_ID	ORDER_ID	PRODUCT_ID	ORDER_SIZE	QUANTITY
302	234	230	L	2
304	255	250	L	2
301	223	220	S	1
303	245	230	M	1
305	265	260	S	1

5 rows returned in 0.04 seconds [Download](#)

```
--I created a new table EMPLOYEES
CREATE TABLE EMPLOYEES (
  EMPLOYEE_ID NUMBER(7) CONSTRAINT pk_employee PRIMARY KEY,
  FIRST_NAME_EMP VARCHAR2(20),
  LAST_NAME_EMP VARCHAR2(20),
  MANAGER_ID NUMBER(7),
  CONSTRAINT fk_manager FOREIGN KEY (MANAGER_ID) REFERENCES EMPLOYEES(EMPLOYEE_ID)
);
```

```
--I inserted data in the table "EMPLOYEES"
INSERT INTO EMPLOYEES (EMPLOYEE_ID, FIRST_NAME_EMP, LAST_NAME_EMP, MANAGER_ID) VALUES (1, 'Mariana', 'Popa', NULL);
INSERT INTO EMPLOYEES (EMPLOYEE_ID, FIRST_NAME_EMP, LAST_NAME_EMP, MANAGER_ID) VALUES (2, 'Alexandra', 'Popescu', 1);
INSERT INTO EMPLOYEES (EMPLOYEE_ID, FIRST_NAME_EMP, LAST_NAME_EMP, MANAGER_ID) VALUES (3, 'Stefan', 'Adam', 1);
INSERT INTO EMPLOYEES (EMPLOYEE_ID, FIRST_NAME_EMP, LAST_NAME_EMP, MANAGER_ID) VALUES (4, 'Maria', 'Apostolescu', 2);
INSERT INTO EMPLOYEES (EMPLOYEE_ID, FIRST_NAME_EMP, LAST_NAME_EMP, MANAGER_ID) VALUES (5, 'Alex', 'Stoica', 2);
INSERT INTO EMPLOYEES (EMPLOYEE_ID, FIRST_NAME_EMP, LAST_NAME_EMP, MANAGER_ID) VALUES (6, 'Iulia', 'Nitu', 3);
```

```
--25.Display the employees of the company subordinated
--to the root record- sort by hierarchical level
--using (CONNECT BY PRIOR, LEVEL)
SELECT EMPLOYEE_ID, FIRST_NAME_EMP, LAST_NAME_EMP, MANAGER_ID, LEVEL
FROM EMPLOYEES
CONNECT BY PRIOR EMPLOYEE_ID = MANAGER_ID
START WITH FIRST_NAME_EMP = 'Mariana' AND LAST_NAME_EMP = 'Popa'
ORDER BY LEVEL;
```

Results	Explain	Describe	Saved SQL	History
EMPLOYEE_ID	FIRST_NAME_EMP	LAST_NAME_EMP	MANAGER_ID	LEVEL
1	Mariana	Popa	-	1
3	Stefan	Adam	1	2
2	Alexandra	Popescu	1	2
5	Alex	Stoica	2	3
6	Iulia	Nitu	3	3
4	Maria	Apostolescu	2	3

6 rows returned in 0.05 seconds [Download](#)

```
--26.Display the employees of the company subordinated to the root record
--(USING SYS_CONNECT_BY_PATH)
SELECT EMPLOYEE_ID, FIRST_NAME_EMP, LAST_NAME_EMP, MANAGER_ID,
LEVEL, SYS_CONNECT_BY_PATH(FIRST_NAME_EMP || ' ' || LAST_NAME_EMP, '/') AS EMP_PATH
FROM EMPLOYEES
CONNECT BY PRIOR EMPLOYEE_ID = MANAGER_ID
START WITH FIRST_NAME_EMP = 'Mariana' AND LAST_NAME_EMP = 'Popa';
```

Results					
Explain Describe Saved SQL History					
EMPLOYEE_ID	FIRST_NAME_EMP	LAST_NAME_EMP	MANAGER_ID	LEVEL	EMP_PATH
1	Mariana	Popa	-	1	/Mariana Popa
2	Alexandra	Popescu	1	2	/Mariana Popa/Alexandra Popescu
4	Maria	Apostolescu	2	3	/Mariana Popa/Alexandra Popescu/Maria Apostolescu
5	Alex	Stoica	2	3	/Mariana Popa/Alexandra Popescu/Alex Stoica
3	Stefan	Adam	1	2	/Mariana Popa/Stefan Adam
6	Iulia	Nitu	3	3	/Mariana Popa/Stefan Adam/Iulia Nitu
6 rows returned in 0.03 seconds Download					